



School Management SaaS

Tech Stack & Architecture Recommendation Document

1. Backend Framework: NestJS (TypeScript)

Factor	Why NestJS Wins
Ecosystem Unity	Frontend is Next.js/TypeScript — same language end-to-end means shared types, shared validation schemas (Zod), and a single hiring profile
Enterprise Structure	Built-in modules, guards, interceptors, pipes — perfect for multi-tenant SaaS with complex RBAC (admin, teacher, student, parent, staff)
Queue Support	Native <code>@nestjs/bull</code> integration for handling monthly fee payment spikes via job queues
Microservice-Ready	Supports CQRS, event-driven patterns, and microservice transports without rewriting
ORM	Prisma or TypeORM integrate beautifully — Prisma especially with type-safe queries

Why not the others?

Laravel: Great framework, but introduces PHP — separate ecosystem, separate developers, no type sharing with your Next.js frontend.

Golang: Raw performance is overkill here; your bottleneck will be DB and I/O, not CPU. Go's ecosystem for complex business logic (ORM, validation, RBAC) is less mature, and development velocity will be slower.

NestJS gives you **90% of Go's performance** for this workload with **3x developer productivity**.

2. Database: PostgreSQL

Requirement	PostgreSQL Advantage
Fee Payments	ACID transactions are non-negotiable for financial data. PostgreSQL is gold-standard.
Multi-Tenancy	Row Level Security (RLS) built-in — enforce tenant isolation at the DB level, not just application level.
Complex Queries	Student reports, fee reports, attendance analytics — all relational queries that PostgreSQL excels at.
JSON Flexibility	jsonb columns give you MongoDB-like flexibility inside a relational DB.
Scalability	Connection pooling (PgBouncer), read replicas, partitioning — all mature and battle-tested.

Why not MongoDB? Your data is inherently relational (students → classes → teachers → subjects → exams → grades). Forcing this into documents creates painful joins, data duplication, and consistency nightmares — especially for financial transactions.

Why not MySQL? PostgreSQL has superior JSON support, RLS for multi-tenancy, better indexing options, and a more robust feature set. MySQL is fine, but PostgreSQL is simply better for SaaS.

3. Multi-Tenancy: Schema-per-Tenant (Shared DB)

This is the most important architectural decision. Here are your three options:

Approach	Cost	Isolation	Manageability	Verdict
Separate DB/Server per school	\$\$\$\$\$	Best	Hard to manage & update	Overkill
Shared DB, Separate Schema per school	\$\$\$	Good	Easy to manage	 RECOMMENDED
Shared DB, Shared Schema (tenant_id column)	\$	Weak	Easiest	Risk of data leaks

 Recommended Architecture: Separate PostgreSQL Schema per School

```
PostgreSQL Instance
├─ school_abc_schema
│  ├─ students
│  ├─ teachers
│  ├─ fees
│  └─ ...
├─ school_xyz_schema
│  ├─ students
│  ├─ teachers
│  ├─ fees
│  └─ ...
└─ public (shared schema)
   ├─ schools (tenant registry)
   ├─ plans (subscription tiers)
   └─ global_config
```

Why this works:

- **Complete data isolation** — one school can never accidentally see another school's data
- **Easy backup/restore** — dump a single school's schema independently
- **Easy onboarding** — creating a new school = creating a new schema + running migrations
- **Shared infrastructure** — one PostgreSQL instance, one server, one deployment

- **Individual scaling** — if one school grows huge, migrate their schema to a dedicated DB later
- **Cost effective** — you don't pay for 50 separate servers/databases

Handling Monthly Fee Payment Spikes

- Use **Redis + BullMQ** job queues to process payments asynchronously
- Students submit payment → job enters queue → workers process in background
- Add **PgBouncer** for connection pooling (critical when 3000 students hit the DB simultaneously)
- Even 50 schools × 3000 students = 150K payments spread over several days — a single well-configured PostgreSQL instance handles this easily

4. File Storage: S3-Compatible Object Storage

⚠ **Do NOT store files on your VPS disk.** VPS disk is not scalable, not redundant, and ties your files to a single machine.

Option	Pros	Cons	Cost
Cloudflare R2	Zero egress fees, S3-compatible, CDN built-in	Cloud dependency	~\$0.015/GB/month
MinIO on VPS	Self-hosted, S3-compatible, full control	You manage it, need backup	Cost of extra VPS disk
AWS S3	Industry standard, highly durable	Egress fees add up fast	\$0.023/GB + \$0.09/GB egress

✅ Recommended: Cloudflare R2

- **Zero egress fees** — students/parents downloading certificates, assignments, documents won't cost you per download
- S3-compatible API — uses the same `@aws-sdk/client-s3` package
- Built-in CDN for fast delivery
- Estimated cost for 100 schools: **~\$5-15/month** for storage

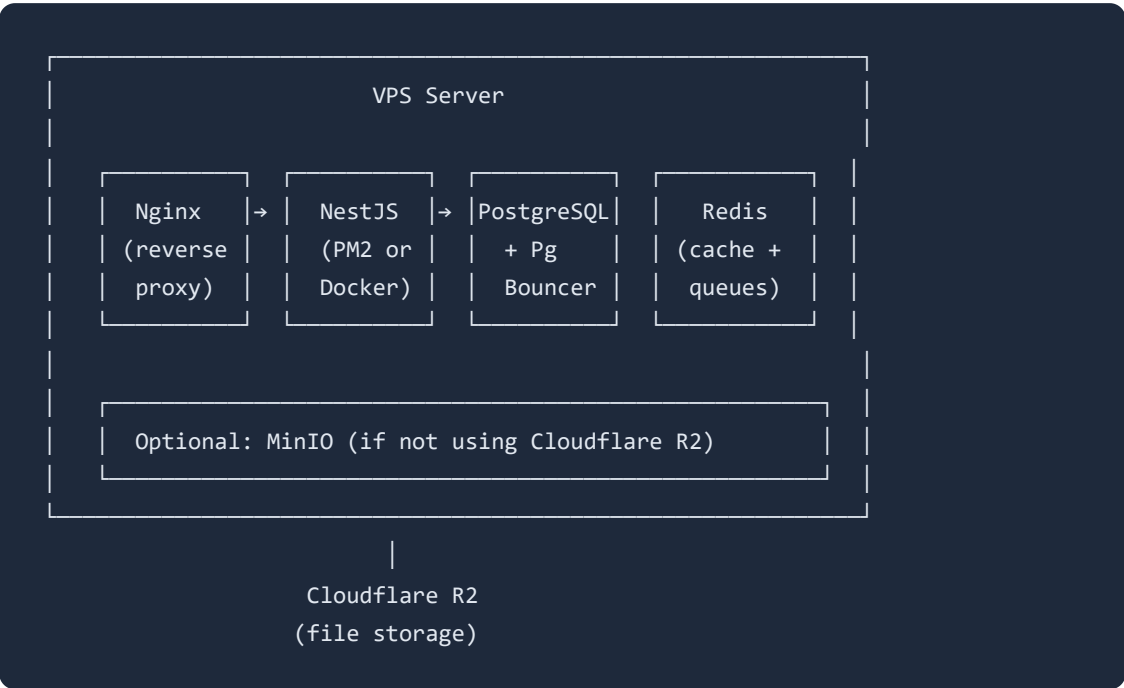
Upload/Download Architecture

Student uploads assignment

- NestJS generates presigned upload URL (R2)
- Frontend uploads directly to R2 (bypasses your server)
- NestJS saves the file reference in PostgreSQL
- Download: NestJS generates presigned download URL
- Frontend downloads directly from R2 (bypasses your server)

This means **your VPS never handles file transfer traffic** — only metadata.

5. VPS Hosting Architecture



Recommended VPS Specs

Schools	RAM	CPU	Storage	Monthly Cost
1–10 schools	4–8 GB	2–4 vCPU	80 GB SSD	\$20–40
10–50 schools	16 GB	4–6 vCPU	160 GB SSD	\$60–100
50–200 schools	32 GB	8 vCPU	320 GB SSD	\$150–200

Providers to consider: Hetzner (best value), DigitalOcean, Vultr, Contabo

6. Final Summary

Decision	Recommendation	Reason
----------	----------------	--------

Backend	NestJS (TypeScript)	Same ecosystem as Next.js, enterprise-grade, queue support
Database	PostgreSQL	ACID for payments, RLS for tenancy, relational data model
Multi-tenancy	Schema-per-tenant (shared DB)	Best balance of isolation, cost, and manageability
File Storage	Cloudflare R2	Zero egress, S3-compatible, cheap, CDN included
Caching / Queues	Redis + BullMQ	Monthly fee payment spikes, session management
Hosting	VPS (Hetzner/DO)	Cost-effective, start small and scale vertically
ORM	Prisma	Type-safe, excellent PostgreSQL support, schema-per-tenant patterns

This architecture will comfortably handle **200+ schools** on a single VPS before you need to think about horizontal scaling.